



Logistic Regression Vs Random Forest Classifier

Last Updated : 23 Jul, 2025

A statistical technique called **logistic regression** is used to solve problems involving binary classification, in which the objective is to predict a binary result (such as yes/no, true/false, or 0/1) based on one or more predictor variables (also known as independent variables, features, or predictors). Based on the values of the predictor variables, logistic regression produces a probability score that indicates the likelihood of the positive class (for example, yes, true, or 1).

A logistic function is used in logistic regression to model the connection between the predictor variables and the binary result. A [probability](#) score between 0 and 1 is produced from the linear combination of predictor variables by the logistic function. A binary prediction can then be made using the altered score that has been threshold. By employing [maximum likelihood estimation](#), which determines the values of the coefficients that maximize the likelihood of the observed data given the model, the logistic regression model's coefficients are calculated.

For binary classification issues, [logistic regression](#) is frequently utilized in industries including health, finance, and marketing. Large volumes of data may be handled, and it is simple and quick. It cannot describe complex linkages and interactions between variables, and it makes the assumption that the predictor variables and the binary outcome have linear relationships.

Difference between Logistic Regression and Random Forest

Now let's take a look at the differences between the Logistic Regression and the Random Forests model in the tabular form. In this way, we will be able to conclude all the necessary points in one place.

S.NO.	Random Forest	Logistic Regression
1.	It is Suitable for both classification and regression problems.	It is Suitable only for binary classification problems.
2.	Makes predictions based on an ensemble of decision trees.	Makes predictions based on a logistic function.
3.	Can handle missing values, outliers, and non-linear relationships.	Assumes linear relationships between the independent and dependent variables.
4.	Does not require feature scaling.	Requires feature scaling for optimal performance.
5.	More accurate and robust compared to individual decision trees.	Less accurate and robust compared to Random Forest but computationally faster.
6.	Can handle high-dimensional data better.	Can handle high-dimensional data but prone to overfitting.
7.	Can model complex relationships between variables.	Limited to modeling linear relationships between variables.

8.	Can handle large amounts of data efficiently.	Can become computationally expensive for large amounts of data.
9.	Can handle imbalanced datasets better.	Prone to biased predictions for imbalanced datasets.
10.	Can be used for feature importance analysis.	Does not provide feature importance analysis.
11.	Can be time-consuming to train.	Quick to train compared to Random Forest.

Features of Logistic Regression:

- **Binary Classification:** When trying to predict one of two outcomes in a binary classification problem, logistic regression is specifically used.
- **Linear Modeling:** Logistic Regression uses a linear combination of the predictor variables to model

Python for Machine Learning

Machine Learning with R

Machine Learning Algorithms

EDA

Math for Machine Le



Sign In

- **Logistic Function:** Using a logistic function, the linear combination of predictor variables is converted into a probability score between 0 and 1, which indicates the likelihood that the class is positive.
- **Maximum Likelihood Estimation:** This method determines the values of the coefficients that maximize the likelihood of the observed data given the model for the logistic regression model's coefficients.
- **Interpretability:** Logistic regression offers a simple and understandable model, where the model's coefficients stand in for each predictor variable's influence on the binary outcome.
- **Computationally Efficient:** Logistic Regression is computationally effective, making it appropriate for use with big datasets.
- **Robust to Outliers:** Logistic regression is comparatively resilient against [outliers](#), which means that a few outlier values in the predictor variables do not have a big impact on the model.
- **Feature Scaling:** For optimal performance, feature scaling—in which the predictor variables are modified to have comparable scales—is necessary for logistic regression.
- **Limitations:** There are several restrictions associated with the use of logistic regression, including the assumption of linear connections between the predictor variables and the binary outcome as well as the incapability to model intricate interrelationships and interactions between variables.

Now let's see an example implementation of logistic regression using the Iris dataset:

```
from sklearn.metrics import accuracy_score
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression

# Load the dataset
iris = load_iris()

# Split the dataset into features and target variable
X = iris.data
y = iris.target

# Split the data into training and testing sets
X_train, X_test, \
y_train, y_test = train_test_split(X, y,
                                   test_size=0.2,
                                   random_state=42)

# Create an instance of the Logistic regression model
lr = LogisticRegression()
```

```
# Fit the model to the training data
lr.fit(X_train, y_train)

# Make predictions on the testing data
y_pred = lr.predict(X_test)

# Calculate the accuracy of the model
accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")

# This code is written by Lavanya Bisht
```

Output:

Accuracy: 1.0

Random Forest

An [ensemble learning technique](#) called Random Forest is applied to both classification and regression issues. A decision tree method of this kind combines the predictions of numerous decision trees, or forests, to arrive at a final prediction.

Each decision tree in [Random Forest](#) is constructed using a unique bootstrap sample of the data and a unique subset of the predictor variables known as a random subspace. As a result, the predictions of the trees are aggregated either by majority vote for classification issues or by average for regression problems. This means that each tree in the forest is unique.

In comparison to individual decision trees, Random Forest offers a number of advantages. By averaging the predictions of several trees rather than depending just on one, it first lowers [overfitting](#). Second, by lowering the variance in the trees' predictions, it improves the model's stability and accuracy. It also offers feature importance, which can be used to pinpoint the model's most crucial predictor variables.

Because of its precision, stability, and capacity for handling massive volumes of data, Random Forest is frequently employed in industries including finance, medicine, and biology. It can manage non-linear correlations between the predictor factors and the outcome and is also quite simple to comprehend. However, it is computationally expensive and can be slow to train and make predictions, especially for large datasets.

Features of Random Forest:

- **Ensemble Learning:** To prevent overfitting and increase accuracy in comparison to individual decision trees, Random Forest uses ensemble learning to integrate the predictions of various decision trees into a single forecast.
- **Bootstrapped Samples:** The Random Forest builds each decision tree using a unique bootstrapped sample of the data, which lowers overfitting and boosts stability.
- **Random Subspaces:** To assist prevent overfitting and boost stability, each decision tree in the Random Forest is constructed using a unique random subset of the predictor variables.
- **Averaging:** A final prediction for classification issues is made by Random Forest by averaging the forecasts of the decision trees. It generates a final forecast for regression issues by averaging the results of the [decision trees](#).
- **Feature Importance:** Random Forest provides feature importance, which shows how each predictor variable affects the model's ability to predict outcomes accurately. The most significant predictor variables in the model can be found using this.
- **Non-Linear Relationships:** Unlike linear regression, which requires linear relationships between the predictor variables and the outcome, the random forest can accommodate non-linear relationships.

- **Handling Missing Data:** By building decision trees using the available data and averaging their predictions, Random Forest can handle missing set in the training data.
- **Costly to Run:** Random forests can take a long time to train and produce accurate predictions, especially for huge datasets.
- **Simple to Read:** Random Forest offers feature importance that may be utilized to understand the connections between the predictor variables and the outcome. It is also comparatively simple to interpret.

Now let's see the example of a random forest using the iris dataset:

```
from sklearn.ensemble import RandomForestClassifier

# Create an instance of the Random Forest classifier
rf = RandomForestClassifier(n_estimators=100,
                           random_state=42)
rf.fit(X_train, y_train)
y_pred = rf.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print(f"Accuracy: {accuracy}")
```

Output:

Accuracy: 1.0

Comment

L lavany...

0

Article Tags: [Machine Learning](#) [python](#)

Explore

Machine Learning Basics

Python for Machine Learning

Feature Engineering

Supervised Learning

Unsupervised Learning

Model Evaluation and Tuning

Advanced Techniques

Machine Learning Practice